LibraUni

KNOWLEDGE PURSUED FREELY

P101 · Unit 3 · Quick reference

Motion graphs and reproducible data work

$$\Delta x = x_2 - x_1 \qquad \Delta t = t_2 - t_1 > 0$$

$$\text{Average velocity} = \frac{\Delta x}{\Delta t} \qquad \text{Average speed} = \frac{\text{total distance}}{\Delta t}$$

$$x_{\text{pred}}(t) = x_{\text{ref}} + v(t - t_{\text{ref}}) \qquad r = x_{\text{observed}} - x_{\text{predicted}}$$

Positions / m · times / s · rates / m/s · residuals / m

Recall	Conventions	State origin, positive direction and tracked point. Clock reading differs from elapsed time.
Recall	Rates	Include stops in elapsed time. Distance needs the path; displacement needs endpoints.
Recall	Model	Fixed coordinate system; constant velocity over the modelled interval. At reference time, return reference position.
Recall	Residual	Observed minus predicted at the same time. Positive residual: observed position on positive side of prediction.
Understand	Graph	Interval gradient gives average velocity. A straight model segment asserts constant velocity; isolated points do not.
Understand	Reach	Interpolation inside the stated data interval; extrapolation outside. Neither guarantees accuracy. Construction agreement is built in.
Consult	Handover	Identified unchanged source; conventions; selection and model choices; saved code; labelled output; actual run/version note.
Consult	Checks	Hand calculation; reference-time and zero-velocity cases; fresh ordered run. Missing is not zero; execution is not physical validation.

Unit 3 · Python lookup

Consult · Previously taught patterns; no new syntax.
Inspect source identity, units, increasing distinct times and expected shape. Check finite clock readings before selecting pairs; inspect an empty selection.

Load a documented CSV

```
from pathlib import Path
import numpy as np
p = Path.cwd() / "data" / "record.csv"
a = np.genfromtxt(p, delimiter=",", skip_header=1,
    dtype=float, filling_values=np.nan,
    ndmin=2, loose=False)
print(a.shape)  # inspect expected rows and columns
```

Select complete pairs

```
t_all = a[:, 0]
x_all = a[:, 1]
keep = np.isfinite(t_all) & np.isfinite(x_all)
t = t_all[keep]
x = x_all[keep]
print(t_all[~keep])  # omitted times; source unchanged
```

Predict and compare

```
def predict(t, t_ref, x_ref, v):
    return x_ref + v * (t - t_ref)
y = predict(t, 20.0, 1.0, 1.0)  # C1 example
r = x - y  # observed minus predicted
```

Plot and save

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot(t, x, "o", label="Available observations")
ax.set_xlabel("Clock reading / s")
ax.set_ylabel("Position / m")
ax.legend()
fig.savefig("positions.png", dpi=160)
plt.show()
```

Record the environment

```
import platform
import matplotlib
print(platform.python_version())
print(np.__version__, matplotlib.__version__)
```

Restart and run all in order; check by hand; record actual versions and outcomes.